Lazy Evaluation with Streams

Delaying work until a value is needed

CS413

Fall 2026

Evaluation can happen now or later

In a strict language, a function's arguments are evaluated before the call. Lazy evaluation delays a computation until its result is requested.

Strict argument

```
choose(42, expensive())
```

Both arguments are evaluated before `choose` runs.

Delayed argument

```
choose(42, thunk(expensive))
```

The second computation can wait until it is needed.

This lecture uses explicit thunks and lazy streams, so the delay is visible in the program rather than built into every expression.

A thunk packages a deferred computation

A thunk is a function with no meaningful input that computes a value later.

```
Lazy[T] = () -> T
```

In LAMBDA, the interface is written using `lazy_make` and `lazy_eval`:

```
deferred : Lazy[sint] =
  lazy_make(lam (u: unit) => 6 * 7)
answer = lazy_eval(deferred)
```

Creating `deferred` stores the computation; `lazy_eval` requests its value. In this runtime, a lazy value caches its first result.

A lazy stream delays the next cell

A finite list stores its whole spine. A stream can store one cell now and delay the rest.

```
Stcn[T] = stcn_nil()
        | stcn_cons(head: T, tail: Strm[T])
Strm[T] = Lazy[Stcn[T]]
```

- `stcn_nil` marks the end.
- `stcn_cons(x, xs)` stores a head and a delayed stream tail.
- The outer `Lazy` also delays inspection of the current cell.

This is the course library's explicit representation of a lazy list.

Constructing a stream delays its constructor

The library wraps a stream cell in a thunk:

```
strm_nil[T] : unit -> Strm[T] = lam (u) =>
  lazy_make(lam (v) => stcn_nil())

strm_cons[T] : (T, Strm[T]) -> Strm[T] = lam (p) =>
  lazy_make(lam (u) => stcn_cons(p.0, p.1))
```

- Calling `strm_cons` creates a stream value without inspecting its cell.
- `strm_head(xs)` forces the outer thunk, then reads the head.
- `strm_tail(xs)` gives the tail stream; its cell stays delayed.

Operations such as `strm_head` need a nonempty stream.

Demand propagates only as far as the consumer asks

Suppose the stream has cells `1, 2, 3, ...`. To get the first head:

1. Force the outer lazy value to reveal a `stcn_cons`.
2. Read its head, `1`.
3. Leave its tail untouched.

```
strm_head(xs)               # forces one cell
strm_head(strm_tail(xs))    # forces two cells
```

A consumer controls how much of a stream gets evaluated. Reusing a forced lazy cell reuses its cached result in this implementation.

A stream can describe an unbounded sequence

Define a stream of integers starting at `n`; the recursive call sits inside a thunk, so it does not run while the current cell is built:

```
naturals_from : sint -> Strm[sint] = fix from(n) =>
  strm_cons(n,
    lazy_make(lam (u: unit) => lazy_eval(from(n + 1))))
```

- Constructing `naturals_from(0)` returns after building its first cell.
- The tail computation builds the next cell only when demanded.
- The stream has no final cell, but any finite prefix can be observed.

`strm_range(lo, hi)` in the library is the corresponding finite example.

Mapping transforms cells as they are requested

`strm_map` delays each output cell and maps the head when that cell is forced:

```
strm_map(xs, lam (x) => x * 2)
```

For a source stream `1, 2, 3, ...`, its result describes `2, 4, 6, ...`. Creating the mapped stream does not traverse the source.

```
strm_to_list(
  strm_take(strm_map(naturals_from(1), lam (x) => x * 2), 4))
# [2, 4, 6, 8]
```

`take` makes the finite demand explicit before conversion to a list.

Filtering may search ahead for the next output

`strm_filter(xs, p)` skips input cells until it finds one satisfying `p`.

```
strm_filter(naturals_from(0), lam (n) => n > 5)
```

Asking for the first output forces inputs 0 through 6. Asking only whether an earlier cell matches can stop sooner:

```
strm_exists(xs, lam (n) => n == 3)
```

`strm_exists` returns as soon as it finds a match. If the predicate never matches an infinite stream, finding the first filtered cell never finishes.

The sieve generates primes on demand

Start with the stream `2, 3, 4, 5, ...`. Keep its head as the next prime, filter multiples of that prime from the tail, then sieve the result:

```
sieve(xs) =
  let p = strm_head(xs)
  in strm_cons(p,
       lazy_make(lam (u: unit) =>
         lazy_eval(sieve(strm_filter(
           strm_tail(xs), lam (n) => n mod p != 0)))))

primes = sieve(naturals_from(2))
```

- The first output is 2; its delayed tail removes multiples of 2.
- The next head is 3; the next delayed tail removes multiples of 3.
- Taking a finite prefix forces only enough filtering and sieving to produce it.

`strm_filter` here denotes the library filter; the recursive call is delayed inside the tail thunk.

Lazy composition avoids intermediate lists

A pipeline can describe a sequence of transformations:

source stream $\longrightarrow$ map $\longrightarrow$ filter $\longrightarrow$ take

- Each stage produces a stream whose later cells remain delayed.
- The consumer's request pulls values through the pipeline as needed.
- No full intermediate list is required.

Laziness can save work and support infinite sources. It does not promise that every pipeline uses less memory or runs faster; retained thunks can keep earlier data reachable.

Short-circuit consumers need only a prefix

`strm_exists` stops at the first matching item; `strm_forall` stops at the first item that fails the predicate.

```
strm_exists(xs, lam (x) => x == 1)
```

If the first cell of `xs` is 1, the result is `true`; the consumer need not inspect any later cells. A delayed tail could diverge or raise an error, and this result would still be available.

Key idea

A lazy program can avoid computations that its result does not depend on.

Laziness has costs and limits

- A thunk needs a closure and may retain the values it captures.
- Memoization avoids recomputing a forced cell, but cached values occupy memory.
- A consumer that asks for the whole stream must still do all that work.
- A nonproductive definition may loop before producing its first cell.

Ask two questions when evaluating a lazy pipeline: which cells are forced, and which values remain reachable afterward?

Summary

- Lazy evaluation delays work until a consumer demands a value.
- A thunk represents a deferred computation; this LAMBDA runtime memoizes its result.
- A stream is a lazy cell whose tail is another stream.
- Mapping and filtering can operate on unbounded inputs, while consumers such as `take` and `exists` bound demand.

Quick check

In `strm_exists(xs, p)`, why might it be incorrect to force the tail before testing the head?